Trivia Game — Project Summary

Overview

A multiplayer trivia game built on a **client-server TCP socket architecture**. Players connect via a terminal client, authenticate, then play single-player or team-based multiplayer games. All data is persisted in JSON files — no external database or libraries used.

Architecture

```
Client (ClientMain + ServerListener)
        |  TCP Socket
Server (ServerMain → ClientHandler per connection)
        |
   GameManager / GameSession / SinglePlayerGame
        |
   JSON files (users, scores, questions, answers)
```

- **ServerMain** — starts a `ServerSocket`, accepts connections, spawns a `ClientHandler` thread per client via `ExecutorService`.
- **ClientHandler** — runs on its own thread, handles the full lifecycle for one connected player (auth → menu → game).
- **ClientMain** — connects to the server, starts a `ServerListener` thread to print incoming messages, and reads user input in a loop.

Threading

Where	Thread	Purpose
`ServerMain`	`ExecutorService` thread pool	One thread per connected client
`ClientMain`	`ServerListener` thread	Reads server messages while main thread reads user input
`SinglePlayerGame`	Timer thread (daemon)	Broadcasts countdown per question; interrupted when player answers
`GameSession`	Timer thread per	Runs countdown, fires `evaluateResults()`, advances to next

`GameSession`	question	`evaluateResults()`, advances to next question
`GameSession.start()`	Startup thread	2-second delay before first question so all clients are ready

Key point: the **client's TCP read thread** (`ClientHandler`) is blocked in `waitForGame()` during multiplayer. Game messages are pushed to it via `user.sendMessage()` from the session's timer thread — this is the cross-thread communication pattern used throughout.

Game Flow

Single Player

1. Login → choose category, difficulty, number of questions
2. Questions broadcast one by one with a countdown timer
3. Player types `A/B/C/D` directly; timer thread is interrupted on answer
4. Score saved to `scores.json` with full question detail and `"mode": "single"`

Multiplayer

1. Player creates a team (sets category, difficulty, max questions) or joins an existing one by name only
2. `GameManager` holds a waiting pool keyed by `category|difficulty|maxQuestions`
3. When **two full compatible teams** exist, a `GameSession` is auto-created and started — no manual start command
4. `GameSession` broadcasts questions to all players, collects answers via `AnswerManager`, evaluates after timer expires
5. Final scoreboard shows each player's score + per-question breakdown, then team totals and winner
6. Score saved with `"mode": "multiplayer"`

Design Decisions & Assumptions

- **No GameRoom** — rooms were removed entirely. Teams are the only grouping unit. Matchmaking is automatic when two compatible full teams exist.
- **maxPlayersPerTeam from config** — team size is not hardcoded; it's read from `data/config.json` (`maxPlayersPerTeam`).
- **Answer format** — players type bare `A/B/C/D` (or `ANSWER A`). The server normalises to

uppercase before matching.

- **AnswerManager uses sessionId** — each `GameSession` has a UUID. Answers are stored with that ID so results for one game don't bleed into another.
- **No external JSON library** — `JsonUtil.java` is a hand-written minimal JSON parser/writer to avoid dependencies.
- **Score history stores full detail** — each history entry has `playedAt`, `mode`, `category`, `difficulty`, `totalScore`, and a per-question list with the player's answer, correct answer, and result.
- **Scores are never deleted** — `points` resets to 0 after each game (moved to history), but history accumulates across sessions.

Data Files

File	Purpose
`users.json`	Registered users (name, username, hashed password)
`questions.json`	58 questions across 5 categories × 3 difficulties
`scores.json`	Per-user score history with full game records
`config.json`	`questionTimeSeconds`, `maxPlayersPerTeam`, team limits

How to Run

```
# Compile
javac -cp . models/*.java server/*.java client/*.java

# Start server
java server.ServerMain

# Start client (separate terminal per player)
java client.ClientMain
```